Sprint 1

PROJECT ENVIRONMENT

The project environment utilizes MySQL 8.0 as the relational database management system with MyDQL Workbench 8.0 CE for database design, development, and management. The development environment runs on a local machine with Windows 11 operating system, providing a stable platform for database implementation, query testing, and ER modeling. MySQL Workbench provides the visual interface for creating ER diagrams, writing SQL queries, and managing database connections.

HIGH LEVEL REQUIREMENTS

Initial user roles

User Role	**Description**
Child Participant	A child who participates in cooking sessions and has specific dietary needs and reading levels
Session Facilitator	An adult who organizes and leads cooking sessions, tracks participant engagement, and session outcomes
Content Manager	Administrator who manages book collections, recipes, and thematic content relationships

Initial user story descriptions

Story ID	**Story description**
US1	As a content manager I want to associate recipes with specific books so that cooking activities align with literary themes
US2	As a session facilitator I want to track which children participate in each cooking session so that I can monitor engagement and preferences

US3	As a child participant I want to see which cooking sessions I can join based on my age and dietary needs so that I can participate safely and enjoyably

CONCEPTUAL DESIGN

Include your detailed conceptual design here. Use the format shown below.

Entity: **Books**
Attributes:
- Title
- Author
- Genre
- Age group
- ISBN (PK)

Entity: **Recipes**
Attributes:
- Recipe ID (PK)
- Recipe name
- Instructions
- Difficulty level
- Prep time
- Associated book ISBN (FK) -> Books.ISBN

Entity: **User**
Attributes:
- ChildID (PK)
- Child’s name
- Age
- Reading level
- Dietary restrictions

Entity: **Cooking_Sessions**
Attributes:
- Session ID (PK)
- Session date
- Recipe made (FK) -> Recipes.RescipeID
- Adult in charge

- Rating

Entity: **Themes**
Attributes:
- Theme_ID (PK)
- Theme_Name
- Description

Entity: **Ingredients**
Attributes:
- Ingredient ID (PK)
- Ingredient Name
- Category
- Common Allergens

Relationship: **Books** contains **Recipes**
Cardinality: One to Many
Participation:
    Book has partial participation
    Recipe has total participation

Relationship: **Cooking_Sessions** uses **Recipes**
Cardinality: Many to One
Participation:
    Cooking_Sessions has total participation
    Recipe has partial participation

Relationship: **User** participates_in **Cooking_Session**
Cardinality: Many to Many
Participation:
    User has partial participation
    Cooking_Session has partial participation

Relationship: **Books** categorized_by **Themes**
Cardinality: Many to Many
Participation:
    Book has partial participation
    Theme has partial participation

Relationship: **Recipes** requires **Ingredients**
Cardinality: Many to Many
Participation:
Recipes has total participation
Ingredients has partial participation

LOGICAL DESIGN

Include your logical design here. Use the format shown below.

Entity: **Books**
Columns:
- ISBN (VARCHAR(13), PK)
- Title (VARCHAR(255))
- Author (VARCHAR(255))
- Genre (VARCHAR(100))
- Age_group (VARCHAR(255))

Highest normalization level: BCNF

Entity: **Recipes**
Columns:
- Recipe_ID (INT, PK)
- Recipe_name (VARCHAR(255))
- Instructions (TEXT)
- Difficulty_level (VARCHAR(50))
- Prep_time (INT)
- Associated_book_ISBN (VARCHAR(13), FK -> Books.ISBN)

Highest normalization level: BCNF

Entity: **User**
Columns:
- ChildID (INT, PK)
- Child_name (VARCHAR(255))
- Age (INT)
- Reading_level (VARCHAR(50))
- Dietary_restrictions (VARCHAR(255))

Highest normalization level: BCNF

Entity: **Cooking_Sessions**
Columns:
- Session ID (INT, PK)
- Session_date (DATE)
- Recipe made (INT, FK -> Recipes.Rescipe_ID)
- Adult_in_charge (VARCHAR(255))
- Rating (INT)

  Highest normalization level: BCNF

Entity: **Themes**
Columns:
- Theme_ID (PK)
- Theme_Name (VARCHAR(255))
- Description (TEXT)

  Highest normalization level: BCNF

Entity: **Ingredients**
Columns:
- Ingredient_ID (INT, PK)
- Ingredient_Name (VARCHAR(255))
- Category (VARCHAR(255))
- Common_Allergens (VARCHAR(255))

  Highest normalization level: BCNF

Entity: **Session_Participants**
Columns:
- Session_ID (INT, FK -> Cooking_Sessions.Session_ID)
- Child_ID (INT, FK -> User.ChildID)
- Composite PK: SessionId + Child ID

  Highest normalization level: BCNF

Entity: **Book_Themes**
Columns:
- ISBN (VARCHAR(13), FK -> Books.ISBN)
- Theme ID (INT, FK -> Themes.Theme_ID)
- Composite PK: ISBN + Theme_ID

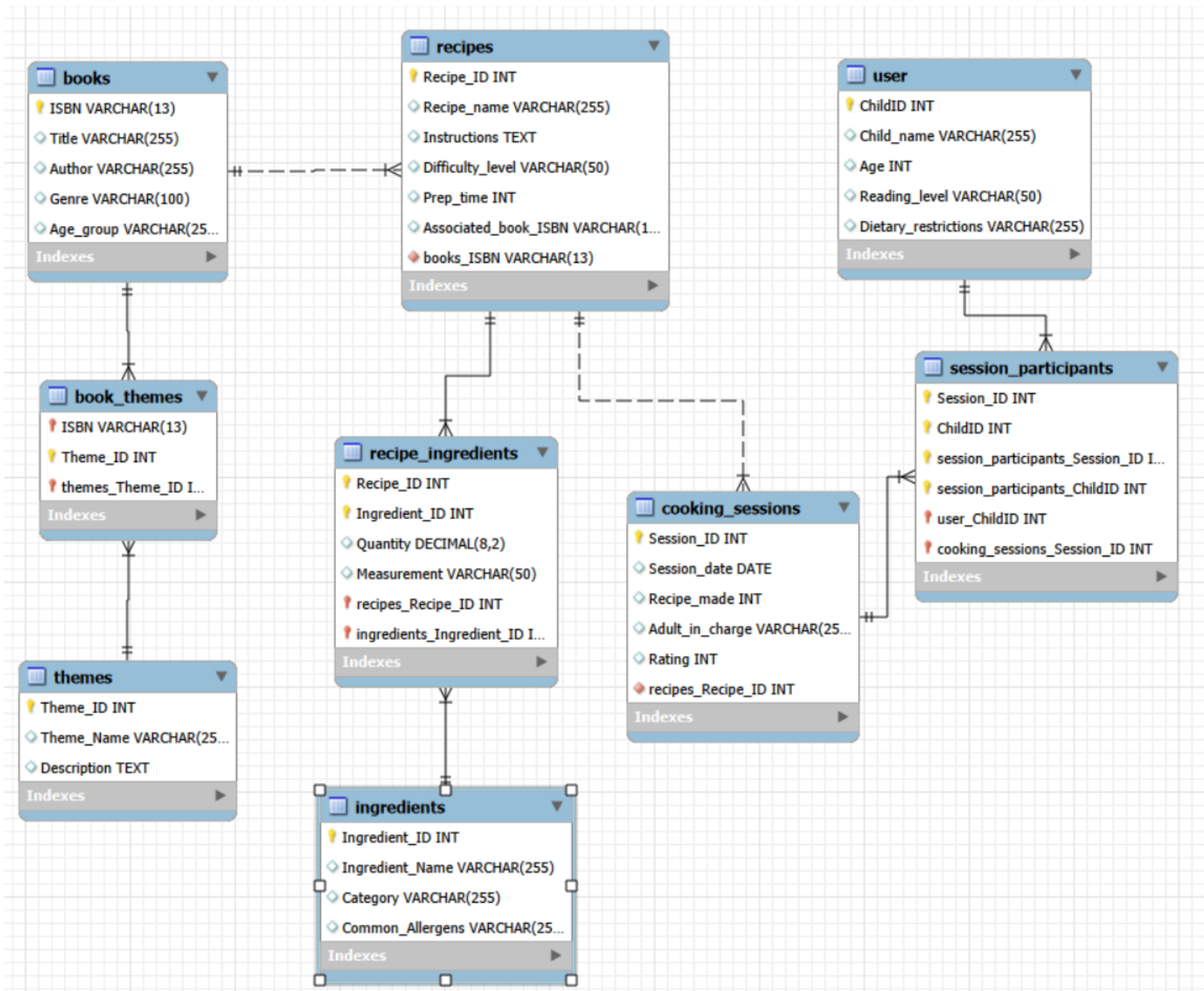
  Highest normalization level: BCNF

Entity: **Recipe_Ingredients**
Columns:
- Recipe_ID (INT, FK -> Recipes.Recipe_ID)
- Ingredient_ID (INT, FK -> Ingredients.Ingredient_ID)

- Quantity (DECIMAL(8,2))
- Measurement (VARCHAR(50))
- Composite PK: Recipe_ID + Ingredient_ID

Highest normalization level: BCNF

ER MODEL



PHYSICAL DATABASE DESIGN

```
CREATE database IF NOT EXISTS little_chef;

USE little_chef;

CREATE TABLE IF NOT EXISTS Books(
ISBN VARCHAR(15) PRIMARY KEY,
```

```
Title VARCHAR(255),
Author VARCHAR(255),
Genre VARCHAR(100),
Age_group VARCHAR(255)
);

CREATE table IF NOT EXISTS Recipes(
Recipe_ID INT primary key,
Recipe_name VARCHAR(255),
Instructions TEXT,
Difficulty_level VARCHAR(50),
Prep_time INT,
Associated_book_ISBN VARCHAR(13),
foreign key (Associated_book_ISBN) REFERENCES Books(ISBN)

);

CREATE TABLE IF NOT EXISTS User (
    ChildID INT PRIMARY KEY,
    Child_name VARCHAR(255),
    Age INT,
    Reading_level VARCHAR(50),
    Dietary_restrictions VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS Themes (
    Theme_ID INT PRIMARY KEY,
    Theme_Name VARCHAR(255),
    Description TEXT
);

CREATE TABLE IF NOT EXISTS Ingredients (
    Ingredient_ID INT PRIMARY KEY,
    Ingredient_Name VARCHAR(255),
    Category VARCHAR(255),
    Common_Allergens VARCHAR(255)
);
```

```
CREATE TABLE IF NOT EXISTS Cooking_Sessions (
    Session_ID INT PRIMARY KEY,
    Session_date DATE,
    Recipe_made INT,
    Adult_in_charge VARCHAR(255),
    Rating INT,
    FOREIGN KEY (Recipe_made) REFERENCES Recipes(Recipe_ID)
);

CREATE TABLE IF NOT EXISTS Session_Participants (
    Session_ID INT,
    ChildID INT,
    PRIMARY KEY (Session_ID, ChildID),
    FOREIGN KEY (Session_ID) REFERENCES
    Cooking_Sessions(Session_ID),
    FOREIGN KEY (ChildID) REFERENCES User(ChildID)
);

CREATE TABLE IF NOT EXISTS Book_Themes (
    ISBN VARCHAR(13),
    Theme_ID INT,
    PRIMARY KEY (ISBN, Theme_ID),
    FOREIGN KEY (ISBN) REFERENCES Books(ISBN),
    FOREIGN KEY (Theme_ID) REFERENCES Themes(Theme_ID)
);

CREATE TABLE IF NOT EXISTS Recipe_Ingredients (
    Recipe_ID INT,
    Ingredient_ID INT,
    Quantity DECIMAL(8,2),
    Measurement VARCHAR(50),
    PRIMARY KEY (Recipe_ID, Ingredient_ID),
    FOREIGN KEY (Recipe_ID) REFERENCES Recipes(Recipe_ID),
    FOREIGN KEY (Ingredient_ID) REFERENCES
    Ingredients(Ingredient_ID)
);
```

SQL QUERIES

List at least **three complex** SQL queries that perform **data retrievals** relevant to the features chosen in the current sprint. (Therefore, you must first insert data in your tables!)For each query, paste a **screenshot** of the output, as shown through the database management tool.

Query 1

```
use little_chef;

/*Query 1: Find safe recipes for children with nut or dairy allergies.
   Return books with their chosen books, themes, and ingredients */
SELECT
   r.Recipe_ID, -- unique identifier for each recipe
   r.Recipe_name, -- name of the recipe
   r.Difficulty_level, -- easy/medium/hard difficulty rating
   r.Prep_time, -- preparation time in minutes
   b.Title AS Book_Title, -- title of the associated book
   b.Age_group, -- recommended age group for the book
   GROUP_CONCAT(DISTINCT t.Theme_Name SEPARATOR ', ') AS
   Themes, -- all book themes combined into one string
   GROUP_CONCAT(DISTINCT i.Ingredient_Name SEPARATOR ', ') AS
   Ingredients -- all ingredients combined into one string
FROM Recipes r
-- join with Books to get book information for each Recipe
JOIN Books b ON r.Associated_book_ISBN = b.ISBN
-- left join with themes to get all book themes
LEFT JOIN Book_Themes bt ON b.ISBN = bt.ISBN
LEFT JOIN Themes t ON bt.Theme_ID = t.Theme_ID
-- join with ingredients to get all recipe ingredients
JOIN Recipe_Ingredients ri ON r.Recipe_ID = ri.Recipe_ID
JOIN Ingredients i ON ri.Ingredient_ID = i.Ingredient_ID
-- filter out recipes that contain nuts or dairy allergens
WHERE r.Recipe_ID NOT IN (
   SELECT DISTINCT ri2.Recipe_ID
   FROM Recipe_Ingredients ri2
   JOIN Ingredients i2 ON ri2.Ingredient_ID = i2.Ingredient_ID
   WHERE i2.Common_Allergens LIKE '%nuts%'
     OR i2.Common_Allergens LIKE '%dairy%'
```

```
)
-- group by recipe
GROUP BY r.Recipe_ID
-- sort by difficulty then prep time
ORDER BY r.Difficulty_level, r.Prep_time;
```

Recipe_ID	Recipe_name	Difficulty_level	Prep_time	Book_Title	Age_group	Themes	Ingredients
1	Caterpillar Fruit Salad	Easy	15	The Very Hungry Caterpillar	3-5	Animals, Food	Apple, Banana, Strawberry
3	Magic Wand Cookies	Easy	30	Harry Potter and the Sorcerer's Stone	9-12	Adventure, Fantasy	Flour, Sugar
5	Green Eggs Scramble	Medium	20	Green Eggs and Ham	4-7	Animals, Food	Eggs, Ham, Spinach

Query 2

```
use little_chef;

/*Query 2: Analyze each child's engagement level, session history, and preferences
to help facilitators understand participation patterns and tailor future sessions */

SELECT
    u.ChildID,                    -- Unique identifier for each child
    u.Child_name,                 -- Full name of the child
    u.Age,                        -- Current age of the child
    u.Dietary_restrictions,       -- Any food allergies or dietary limitations
    COUNT(DISTINCT sp.Session_ID) AS Total_Sessions_Attended,  -- How many unique sessions the child joined
    COUNT(DISTINCT r.Recipe_ID) AS Unique_Recipes_Tried,       -- How many different recipes they've experienced
    AVG(cs.Rating) AS Average_Session_Rating,  -- Average rating of sessions they attended (1-5 scale)
    -- Engagement classification to understand frequency of the childs use
    CASE
        WHEN COUNT(DISTINCT sp.Session_ID) >= 3 THEN 'High Engagement'
```

```sql
        WHEN COUNT(DISTINCT sp.Session_ID) = 2 THEN 'Medium Engagement'
        ELSE 'Low Engagement'
    END AS Engagement_Level  -- Categorizes children based on participation frequency

FROM User u
-- using left join with user include all children not just children who have had at least 1 session
-- use filter later to view only children who are active
LEFT JOIN Session_Participants sp ON u.ChildID = sp.ChildID
LEFT JOIN Cooking_Sessions cs ON sp.Session_ID = cs.Session_ID
LEFT JOIN Recipes r ON cs.Recipe_made = r.Recipe_ID

GROUP BY u.ChildID -- use group by childid to view each childs total engagement in a single row
HAVING Total_Sessions_Attended > 0 -- filter out children who are not active
ORDER BY Total_Sessions_Attended DESC, Average_Session_Rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ChildID	Child_name	Age	Dietary_restrictions	Total_Sessions_Attended	Unique_Recipes_Tried	Average_Session_Rating	Engagement_Level
2	Liam Chen	6	Dairy-free	3	3	4.3333	High Engagement
1	Emma Johnson	7	None	2	2	5.0000	Medium Engagement
3	Sophia Martinez	8	Nut allergy	2	2	5.0000	Medium Engagement
5	Olivia Brown	7	None	2	2	4.5000	Medium Engagement
4	Noah Williams	5	Gluten-free	2	2	4.0000	Medium Engagement

Result 4

Query 3

```
use little_chef;

/*Query 3: Show which recipes are most popular and their average
    ratings*/

SELECT
    r.Recipe_ID,
    r.Recipe_name,
    r.Difficulty_level,
    r.Prep_time,
    b.Title AS Book_Title,
    COUNT(cs.Session_ID) AS Times_Used, -- Count how many times this
    recipe has been used in sessions
    ROUND(AVG(cs.Rating), 1) AS Average_Rating, -- Calculate average
    rating for this recipe across all sessions
    COUNT(DISTINCT sp.ChildID) AS Unique_Children_Reached -- Count
    how many different children have tried this recipe

FROM Recipes r
JOIN Books b ON r.Associated_book_ISBN = b.ISBN
LEFT JOIN Cooking_Sessions cs ON r.Recipe_ID = cs.Recipe_made
LEFT JOIN Session_Participants sp ON cs.Session_ID = sp.Session_ID

-- Group by recipe
GROUP BY r.Recipe_ID, r.Recipe_name, r.Difficulty_level, r.Prep_time,
    b.Title

-- Only show recipes that have been used at least once
HAVING Times_Used > 0
-- Order by most popular and highest rated recipes first
ORDER BY Times_Used DESC, Average_Rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Recipe_ID	Recipe_name	Difficulty_level	Prep_time	Book_Title	Times_Used	Average_Rating	Unique_Children_Reached
1	Caterpillar Fruit Salad	Easy	15	The Very Hungry Caterpillar	3	5.0	3
3	Magic Wand Cookies	Easy	30	Harry Potter and the Sorcerer's Stone	3	5.0	3
2	Charlotte's Web Pancakes	Medium	25	Charlotte's Web	3	4.0	3
4	Snowy Day Hot Cocoa	Easy	10	The Snowy Day	2	4.0	2

Result 2